FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

PROJECT REPORT

AI PUZZLE ESCAPE

A PROJECT REPORT

Submitted by

DURKASRI SM – 240701132

GAYATHRI MOHANRAAJ – 240701143



October, 2026

BONAFIDE CERTIFICATE

Certified that this project report “AI PUZZLE ESCAPE” is the bonafide work of “DURKASRI SM (240701132) and GAYATHRI MOHANRAAJ (240701143)” who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Mrs. B. Deepa

Assistant Professor (SG)

Department of Computer Science and Engineering

Rajalakshmi Engineering College

Submitted for the Practical Examination held on

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	1
1	INTRODUCTION	2
2	LITERATURE REVIEW	4
3	PROPOSED SYSTEM	6
4	MODULES DESCRIPTION	13
5	IMPLEMENTATION AND RESULTS	18
6	CONCLUSION AND FUTURE WORK	26
7	REFERENCES	27

TABLE OF FIGURES

FIGURE NO	TITLE	PAGE NO
Fig. 3.1	System architecture of AI Puzzle Escape	6
Fig. 3.2	Flowchart of the generate-and-verify puzzle builder	10
Fig. 3.3	Illustrative backtracking search tree	11
Fig. 4.1	Module overview of the system	13
Fig. 4.2	Adaptive difficulty decision flow	15
Fig. 4.3	How the three puzzle items form the final answer	16
Fig. 5.1	Start screen of AI Puzzle Escape	19
Fig. 5.2	AI Explanation overlay	20
Fig. 5.3	Puzzle 1: Number Lock with AI Status panel	21
Fig. 5.4	Puzzle 2: Object Arrangement	22
Fig. 5.5	Puzzle 3: Logic Doors after a wrong choice	22
Fig. 5.6	Final Escape stage with collected inventory	23
Fig. 5.7	Victory screen	23
Fig. 5.8	Measured clues and backtracks per difficulty level	24

TABLE OF TABLES

TABLE NUMBER	TITLE	PAGE NO
Table 2.1	Comparison of approaches	5
Table 3.1	Technology stack	7
Table 3.2	Game state variables	7
Table 3.3	CSP formulation of the three puzzles	8
Table 3.4	Difficulty levels	9
Table 4.1	Clue types generated for each puzzle	14
Table 4.2	Adaptive difficulty and support rules	15
Table 5.1	Software environment	18
Table 5.2	Experiment configuration	18
Table 5.3	Number Lock results (200 puzzles per level)	23
Table 5.4	Object Arrangement results (200 puzzles per level)	24
Table 5.5	Logic Doors results (200 puzzles)	24

ABSTRACT

Most puzzle games use puzzles that a designer wrote by hand and whose answers are stored inside the program. Such puzzles are predictable, their fairness cannot be checked automatically, and their difficulty does not change with the player. This project presents AI Puzzle Escape, a browser-based escape-room game in which an artificial intelligence engine builds every puzzle while the game is running. Each puzzle is modelled as a Constraint Satisfaction Problem (CSP) with variables, domains and constraints, and it is solved with backtracking search. To create a puzzle, the engine first chooses a random secret solution by backtracking, then produces clues that are true for that secret and adds them one at a time while counting how many solutions remain. It stops when exactly one solution is left, so every puzzle has a single correct answer. The same constraint checker validates the answers typed by the player, so no answer has to be stored.

The game has four stages: a Number Lock, an Object Arrangement, Logic Doors in which exactly one sign tells the truth, and a Final Escape that combines the three items collected. An adaptive difficulty module observes the time, attempts and hints used in each puzzle and changes the number of extra clues given in the next one, and a struggle detector unlocks one extra hint for a player who is stuck. An AI Status panel shows the number of backtracks and constraint checks live, and an AI Explanation page describes the method to the player. The game is written in HTML, CSS and JavaScript and runs completely in the browser without a server or database. In an experiment with 200 generated puzzles for each puzzle type and difficulty level (1,400 puzzles in total), every puzzle had exactly one solution and its stored answer satisfied every clue. The mean generation time was about 1 ms or less, with a largest single generation time of 14.5 ms.

Keywords: constraint satisfaction, backtracking, puzzle generation, adaptive difficulty, escape room, JavaScript.

CHAPTER 1
INTRODUCTION

1.1 Background

An escape room is a game in which players are locked in a room and must solve a series of puzzles to find the way out. Items found in one puzzle are often needed in a later one, so the puzzles form a chain that ends in a final code. Escape-room ideas have moved into computer games, where the room is a screen and the locks are digital.

In most such games a designer writes every puzzle by hand and stores the answer in the program. This has three weaknesses. First, the game is the same every time, so it can be solved by memory. Second, the designer must check by hand that each puzzle has one and only one correct answer; a careless clue can allow two answers or none. Third, the difficulty is fixed, so it is too easy for some players and too hard for others.

Puzzles of this kind are very close to a classic topic of artificial intelligence, the Constraint Satisfaction Problem (CSP). In a CSP a solution is an assignment of values to variables that obeys a set of constraints. Sudoku, for example, is a CSP (Simonis, 2005). The Foundations of Artificial Intelligence course introduces CSPs and backtracking search, and this project uses them in a practical, playable application: the AI both creates the puzzle and checks the player's answer.

1.2 Problem Statement

Given a puzzle type that is described by variables, a domain of values for each variable and a set of base rules, the system must create a new puzzle with exactly one correct solution each time the game is played. It must show the clues in readable language, check every answer typed by the player, adapt the difficulty to the player's performance, and show how the AI reached its results. All of this must run in an ordinary web browser without installing anything.

1.3 Objectives

- To model each puzzle as a CSP and solve it with backtracking search.
- To generate puzzles automatically at run time so that no answer is hard-coded.
- To guarantee that every puzzle has exactly one solution by counting the solutions that remain after each clue.
- To validate the player's answers with the same constraint checker that the AI uses.
- To adapt the difficulty using the time, attempts and hints of the player, and to give an extra hint to a player who is struggling.

- To make the AI visible through live counters of backtracks and constraint checks, an activity log and an explanation page.
- To provide a complete four-stage escape game with an inventory, timer, hints and a victory screen.

1.4 Scope and Contributions

The system is a single-player game that runs in the browser. It contains three generated puzzle types (Number Lock, Object Arrangement and Logic Doors) and one final stage that combines their results. The main contributions of this work are:

- A small, general CSP engine (`solveCSP`, `backtrack`, `checkConstraints`) that is reused by all three puzzle types.
- A generate-and-verify clue builder that adds true clues until the puzzle has exactly one solution.
- Use of the same constraint checker for generating puzzles and for validating the player's answers.
- A rule-based adaptive difficulty module with struggle detection.
- An AI Status panel and an AI Explanation page that show how the AI works while the game is played.

The rest of this report is organised as follows. Chapter 2 reviews related work. Chapter 3 describes the proposed system, its architecture, the CSP formulation and the algorithm. Chapter 4 explains each module. Chapter 5 presents the experimental setup and results with output screens. Chapter 6 concludes the report and lists future work.

CHAPTER 2
LITERATURE REVIEW

This chapter surveys work on constraint satisfaction and backtracking, puzzles modelled as CSPs, dynamic difficulty adjustment in games and escape-room design, which are the four areas this project draws on.

2.1 Constraint Satisfaction and Backtracking

Russell and Norvig (2020) define a CSP by a set of variables, a domain of values for each variable and a set of constraints. A solution is a complete assignment that satisfies every constraint. Backtracking search builds an assignment one variable at a time and undoes a choice as soon as it breaks a constraint. The idea of backtrack programming was described formally by Golomb and Baumert (1965), and Brailsford, Potts and Smith (1999) survey the algorithms and applications of CSPs.

Many improvements to plain backtracking have been studied. Mackworth (1977) introduced consistency algorithms for networks of relations, which led to constraint propagation. Haralick and Elliott (1980) showed that forward checking combined with a fail-first ordering greatly reduces the search effort. Tsang (1993) and Dechter (2003) give book-length treatments of constraint satisfaction and constraint processing. The Minimum Remaining Values heuristic and forward checking described by these authors are not used in this project; the engine uses plain backtracking that checks a constraint as soon as all of its variables have values. The search spaces of the puzzles are small (at most 10,000 combinations), so this is enough, and the improvements are listed as future work.

2.2 Puzzles as Constraint Satisfaction Problems

Simonis (2005) shows how Sudoku can be written as a constraint problem, with an all-different rule over each row, column and block. The Number Lock and Object Arrangement puzzles of this project use the same all-different rule over their variables. Logic puzzles in which some statements are true and others false have a long tradition (Smullyan, 1978). The Logic Doors puzzle follows this tradition: three signs make statements about the exit, and exactly one of them is true.

2.3 Dynamic Difficulty Adjustment

Hunicke (2005) argues for dynamic difficulty adjustment, in which a game measures the player's performance and changes the challenge while the game is played. The aim follows the idea of flow (Csikszentmihalyi, 1990): a task should be neither so easy that it bores the player nor so hard that it frustrates. This project uses a simple rule-based form of the idea, in which solving time, number of wrong attempts and number of hints decide the next level.

2.4 Escape-Room Games

Nicholson (2015) surveyed escape-room facilities and described their common structure: a team is locked in a themed room, solves linked puzzles, and finds the objects and codes that open the exit. AI Puzzle Escape copies this structure in a digital form: each solved puzzle adds an item to an inventory, and the final stage needs all collected items.

2.5 Summary and Research Gap

Table 2.1 Comparison of approaches

Approach	Strength	Limitation	Use in this project
Hand-written puzzles with stored answers	Designer controls story and difficulty	Fixed and repeatable; answer is stored; fairness checked by hand	Replaced by generation
Random puzzles without verification	Simple; endless variety	May have no solution or many solutions	Not used
CSP generate-and-verify with backtracking	Exactly one solution guaranteed; answers checked by the same rules	Search effort grows with variables and domain size	Core engine
Rule-based difficulty adaptation	Simple and easy to explain	Thresholds are set by hand, not learned	Used
Machine-learned player models	Can use many player features	Needs player data and training	Future work

Many escape-room and puzzle games store their puzzles and answers in the program. Work on CSPs shows how to solve such puzzles but usually treats the solver as a separate tool. This project closes the gap by using one small CSP engine for three jobs inside a playable game: choosing the secret, proving that the clues give a unique answer, and checking what the player enters.

CHAPTER 3

PROPOSED SYSTEM

The proposed system is a single-page web game that runs completely in the browser. The page is described in `index.html`, styled in `style.css`, and all logic is written in `script.js`. There is no server, no database and no external library. All game data is held in memory, so reloading the page starts a new game with new puzzles.

3.1 Architecture Diagram

Figure 3.1 shows the architecture. The code in `script.js` is divided into four parts: the AI engine (Part 1), the puzzle generators (Part 2), the adaptive difficulty module (Part 3) and the game and user interface (Part 4). Each layer calls only the layer below it.

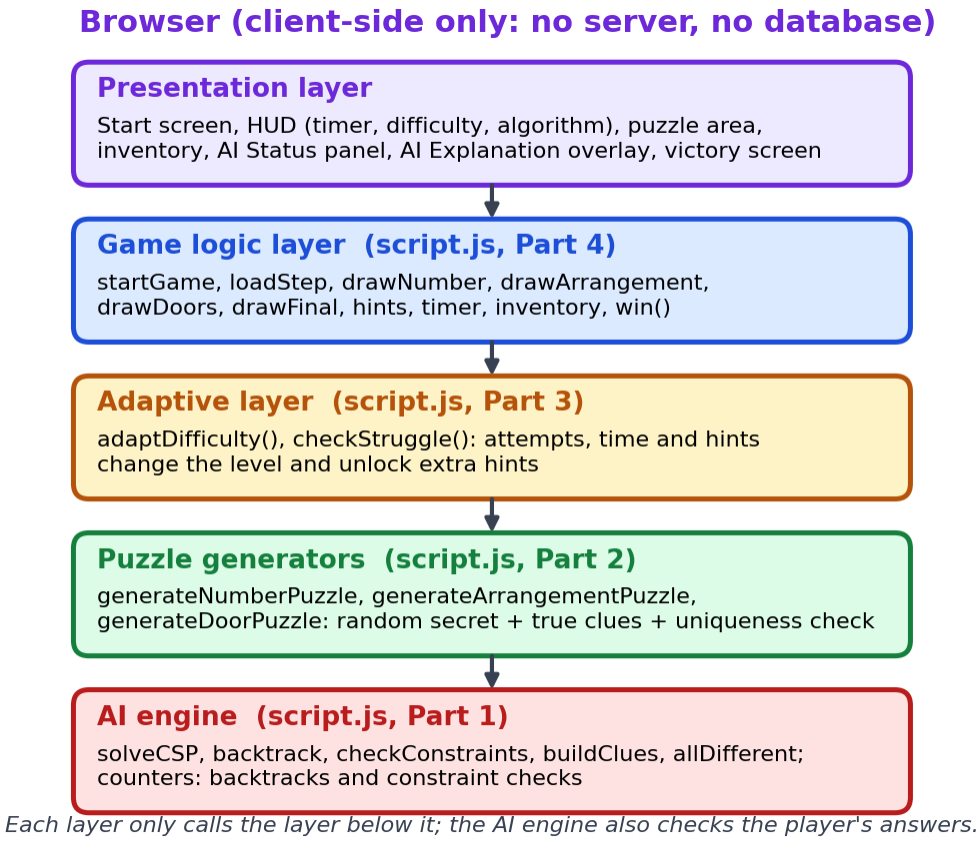


Fig. 3.1 System architecture of AI Puzzle Escape

The architecture has five layers:

- **Presentation layer.** `index.html` and `style.css` define the start screen, the heads-up display (timer, difficulty and algorithm chips), the puzzle area, the inventory, the AI Status panel, the AI Explanation overlay and the victory screen.

- **Game logic layer.** Part 4 of `script.js` keeps the game state and draws each stage: `startGame`, `loadStep`, `drawNumber`, `drawArrangement`, `drawDoors`, `drawFinal` and `win`.
- **Adaptive layer.** Part 3 contains `adaptDifficulty` and `checkStruggle`.
- **Puzzle generators.** Part 2 contains `generateNumberPuzzle`, `generateArrangementPuzzle` and `generateDoorPuzzle`.
- **AI engine.** Part 1 contains the CSP solver (`solveCSP`, `backtrack`), the constraint checker (`checkConstraints`), the clue builder (`buildClues`) and the rule helper `allDifferent`.

Table 3.1 Technology stack

Layer	Technology	Purpose
Structure	HTML5 (index.html)	Start, game, explanation and victory screens
Style	CSS3 (style.css)	Dark theme, grid layout, success and failure animations, confetti, responsive layout below 800 px, reduced-motion support
Logic	JavaScript ES6 (script.js)	AI engine, puzzle generators, adaptive difficulty and game interface
AI method	Custom CSP solver with backtracking	Puzzle generation, uniqueness check and answer validation
Runtime	Any modern web browser	Runs the game; no server, database or library needed

3.2 Game Design and Data Model

The game has four stages, in this order: Number Lock, Object Arrangement, Logic Doors and Final Escape. The first three stages each give the player one item that is stored in the inventory. The final stage asks for a code made from the three items, as shown later in Figure 4.3.

All data is kept in one `game` object and one `stats` object. A puzzle created by a generator is a small object that contains its constraints, the clue text shown to the player, the answer and a list of hints. Table 3.2 lists the game state variables.

Table 3.2 Game state variables

Variable	Meaning
step	Current stage: 0 Number Lock, 1 Objects, 2 Doors, 3 Final Escape
level	Difficulty level: 0 EASY, 1 MEDIUM (start), 2 HARD
attempts	Wrong answers in the current puzzle

Variable	Meaning
hintsUsed, hintIdx	Hints taken in the current puzzle and the index of the next hint
maxHints, extraGiven	Hints allowed (2, or 3 after the struggle bonus) and whether the bonus was given
gStart, pStart	Start time of the game and of the current puzzle
puzzle	The generated puzzle: constraints, clues, answer and hints
items, totalHints	Inventory items collected and hints used in the whole game
stats.backtracks, stats.checks	Counters of rejected values and constraint tests, shown on the AI Status panel

3.3 CSP Formulation

Variables

A variable is a thing that the puzzle asks the player to decide. In the Number Lock the four digits are variables, in the Object Arrangement the four objects are variables (each takes a position), and in the Logic Doors puzzle a single variable stands for the exit door.

Domains

The domain of a variable is the list of values that it may take: the digits 0 to 9, the positions 1 to 4, or the doors A, B and C. The domain lists are shuffled before the search, so the first solution found is a random one.

Constraints

A constraint is an object `{ vars, test, text }`: the variables it involves, a function that returns true or false for their values, and the sentence shown to the player. Because the same object is used for checking and for display, the text of a clue and its rule can never disagree. Table 3.3 summarises the three puzzles.

Table 3.3 CSP formulation of the three puzzles

Puzzle	Variables	Domain	Base rule	Clue constraints	Raw search space
Number Lock	4 digits	0 to 9	All digits different	Even / odd; greater than k; less than k; bigger than another digit; exactly s	10,000 (5,040 with all different)
Object Arrangement	4 objects: Book, Key, Coin, Card	Positions 1 to 4	All positions different	Must be in position; cannot be in position; before; next to; not next to	24

Puzzle	Variables	Domain	Base rule	Clue constraints	Raw search space
Logic Doors	1: the exit door	A, B, C	Exactly one of the three signs is true	Three random signs about the exit	3

The function `checkConstraints` tests a rule only when all of its variables already have values, and it counts every test in `stats.checks`. A partial assignment is therefore accepted as long as no complete rule is broken. The difficulty level changes how many extra clues are added after the puzzle has become unique, as shown in Table 3.4.

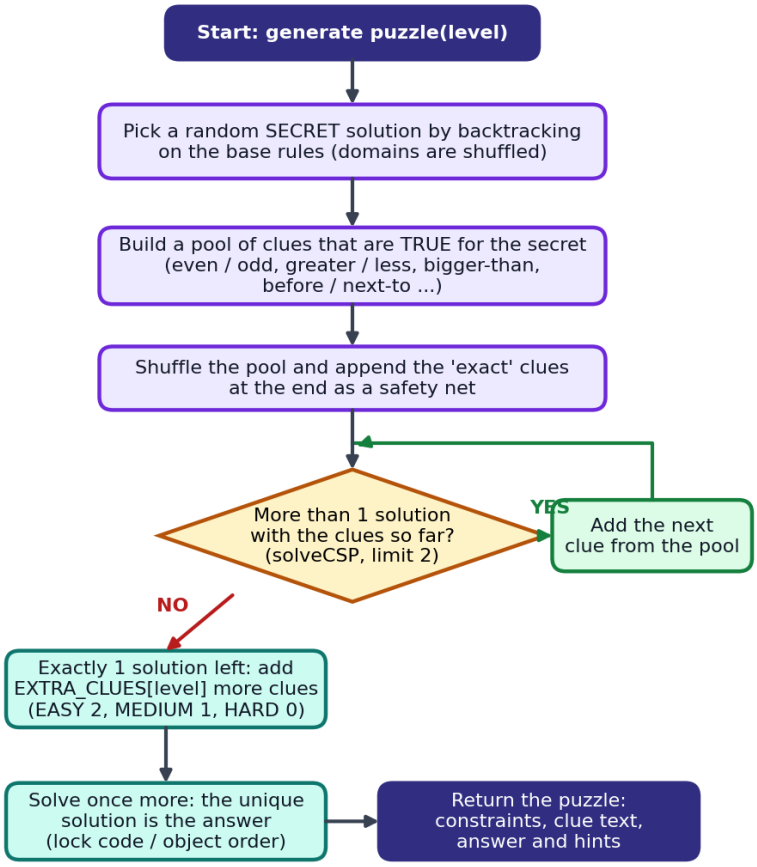
Table 3.4 Difficulty levels

Level	Extra clues after uniqueness	Effect on the next generated puzzle
EASY	2	Two more clues than the minimum needed
MEDIUM (start)	1	One more clue than the minimum needed
HARD	0	Stops as soon as the solution is unique

3.4 CSP Algorithm

The puzzle generator and the solver work together in the following steps, which are also summarised in Figure 3.2.

1. **Choose a random secret.** Run backtracking on the base rules with shuffled domains. The first solution found is the secret.
2. **Build a pool of true clues.** Write clues that are true for the secret (for example “the second digit is even”) and shuffle them. Append “exact” clues (for example “the fourth digit is exactly 6”) at the end of the pool as a safety net.
3. **Add clues until the solution is unique.** Call the solver with a limit of two solutions. While it finds more than one solution, add the next clue of the pool.
4. **Add extra clues.** When exactly one solution is left, add the number of extra clues given by the level (2, 1 or 0).
5. **Solve once more.** The single remaining solution is the answer of the puzzle.
6. **Check the player's answer.** Put the answer into an assignment and call `checkConstraints` with all constraints. Because the solution is unique, an answer that breaks no clue is the intended answer.



The answer is never hard-coded: it is whatever the CSP search finds.

Fig. 3.2 Flowchart of the generate-and-verify puzzle builder

Algorithm 3.1 Backtracking search used by the engine

```
function BACKTRACK(assignment, limit):
    v <- first unassigned variable
    if no such v exists:
        store a copy of assignment as a solution
        return
    for value in domain[v]:
        assignment[v] <- value
        if CHECK_CONSTRAINTS(assignment):   # only complete rules
            BACKTRACK(assignment, limit)
        else:
            backtracks <- backtracks + 1    # value rejected
        delete assignment[v]                # undo the choice
        if number of solutions >= limit:
            return
```

Algorithm 3.2 Building a fair puzzle from a pool of true clues

```
function BUILD_CLUES(base, pool, extra):
    cons <- base;  clues <- empty list;  i <- 0
    while i < size(pool) and SOLVE(cons, limit = 2) has 2 solutions:
        add pool[i] to cons and to clues;  i <- i + 1
    repeat extra times while i < size(pool):
        add pool[i] to cons and to clues;  i <- i + 1
    return cons, clues, SOLVE(cons, limit = 1)[0]
```

Why the search stops at two solutions

The goal is only to know whether the puzzle is unique, so the solver does not need to list every solution. As soon as it finds a second solution it stops, which saves work. Because every clue is true for the secret, the secret always stays a solution, so a solution always exists; the only thing the builder has to achieve is that no other solution remains. The exact clues at the end of the pool guarantee this, since together they fix every variable.

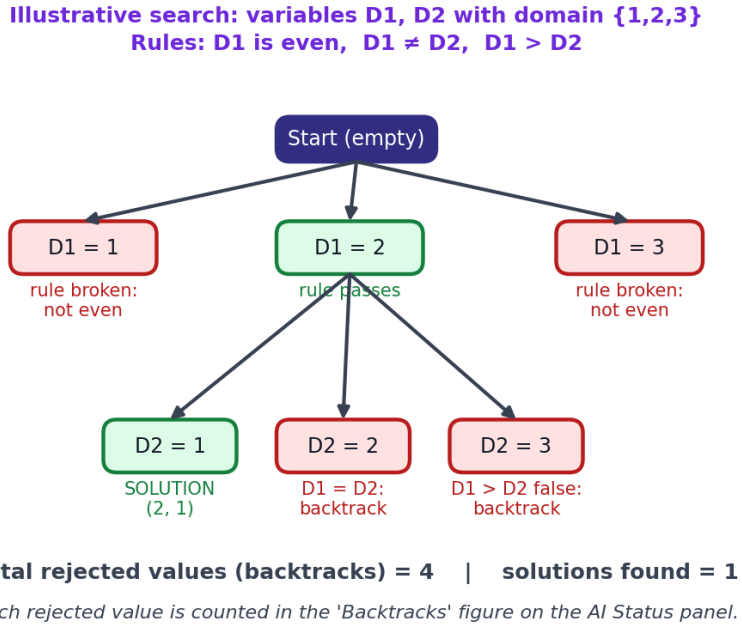


Fig. 3.3 Illustrative backtracking search tree

Complexity

In general CSP solving is NP-hard and backtracking is exponential in the worst case. Here the search spaces are very small (at most 10,000 combinations for the Number Lock, 24 for the Object Arrangement and 3 for the Logic Doors), so plain backtracking is fast enough. Chapter 5 reports the measured counts and times.

3.5 Answer Verification and Explainability

The player's answer is never compared with a stored string. For the Number Lock the typed digits become an assignment `{0:d1, 1:d2, 2:d3, 3:d4}`, for the Object Arrangement the chosen positions become `{object: position}`, and `checkConstraints` is called with the constraints of the puzzle. The Logic Doors puzzle is generated so that exactly one door satisfies the rule “exactly one sign is true”, and the game accepts that door.

The AI is also made visible to the player. The AI Status panel shows the number of backtracks and constraint checks for the current puzzle and the last five AI messages, such as “Generated number lock with CSP + backtracking”. The AI Explanation page explains CSPs, backtracking and adaptive difficulty in plain language.

CHAPTER 4

MODULES DESCRIPTION

The system is divided into seven modules. Figure 4.1 shows how the game interface sits on top of the adaptive, hint, final-stage and explanation modules, which use the puzzle generators and the CSP engine.

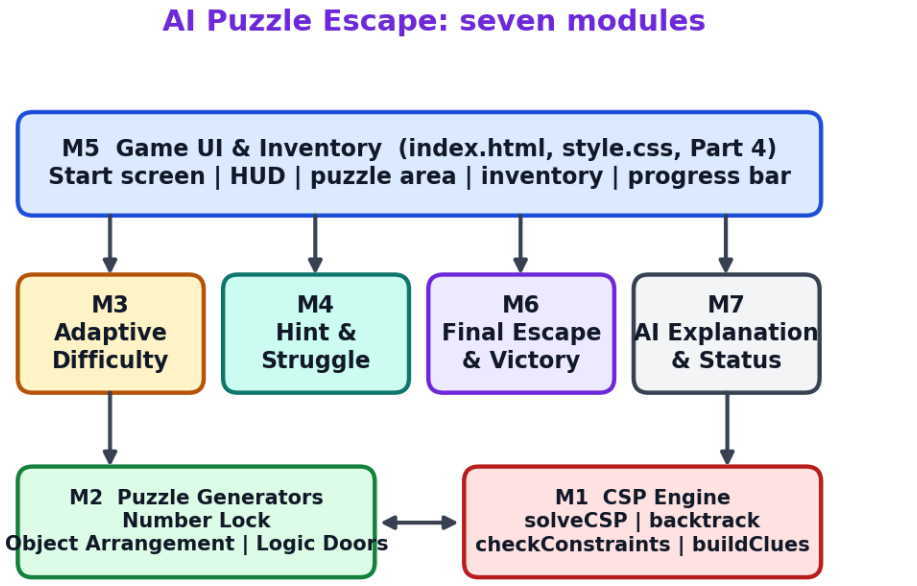


Fig. 4.1 Module overview of the system

4.1 Module 1: CSP Engine

This is the core module (Part 1 of `script.js`). It contains no game rules; it only solves and checks CSPs that are given to it.

- **Input:** a list of variables, a domain for each variable and a list of constraints.
- **Processing:** `backtrack` assigns variables in order and undoes a value when `checkConstraints` finds a broken rule; `solveCSP` collects up to a given number of solutions; `buildClues` adds clues until the solution is unique; `allDifferent` creates the all-different rule for every pair of variables.
- **Output:** a list of solutions (an empty list means the puzzle is impossible) and the counters `stats.backtracks` and `stats.checks`.

4.2 Module 2: Puzzle Generators

Each generator creates a new puzzle with the engine (Part 2 of `script.js`).

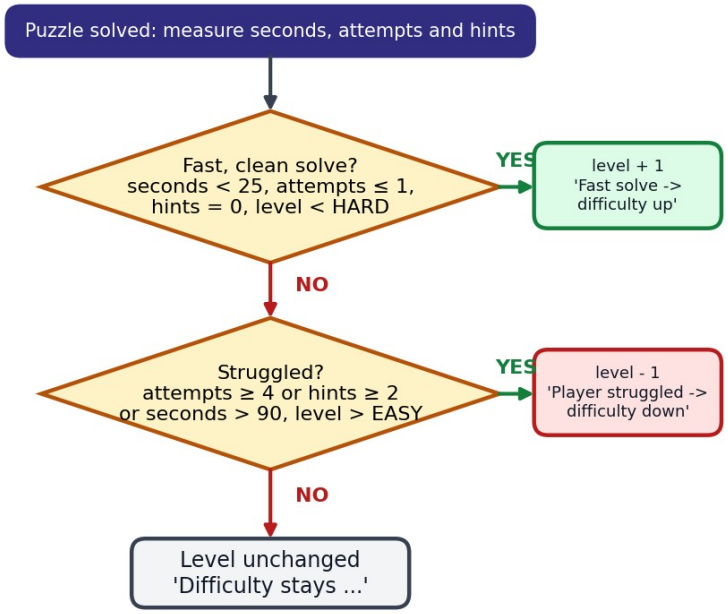
- **Number Lock.** The engine selects a four-digit secret with all digits different. The generator writes clues about parity, comparison with a number, and comparison between two digits, and then calls `buildClues`.
- **Object Arrangement.** The engine selects a random order of Book, Key, Coin and Card in positions 1 to 4. Clues say that an object must or cannot be in a position, that one must appear before another, or that two must or cannot be next to each other.
- **Logic Doors.** The generator repeatedly makes three random signs and keeps the set only if the engine finds exactly one exit door for which exactly one sign is true (up to 500 tries).

Table 4.1 Clue types generated for each puzzle

Puzzle	Example clue sentences produced by the generator
Number Lock	“The second digit is even”; “The first digit is greater than 1”; “The third digit is less than 9”; “The fourth digit is bigger than the second digit”; “The fourth digit is exactly 6” (safety net, used last)
Object Arrangement	“Key must appear before Book”; “Book must be next to Card”; “Book cannot be next to Coin”; “Card cannot be in position 4”; “Key must be in position 2” (safety net, used last)
Logic Doors	“The exit is behind Door B.”; “The exit is NOT behind Door C.”; “The exit is behind Door A or Door C.”

4.3 Module 3: Adaptive Difficulty

After each solved puzzle the module `adaptDifficulty` (Part 3 of `script.js`) uses the solving time, the number of wrong attempts and the number of hints to change the level, as drawn in Figure 4.2. The game starts at MEDIUM. The new level controls the number of extra clues in the next Number Lock or Object Arrangement puzzle; the Logic Doors puzzle does not depend on the level.



The new level sets the extra clues of the NEXT generated puzzle (EASY 2, MEDIUM 1, HARD 0).

Fig. 4.2 Adaptive difficulty decision flow

Table 4.2 Adaptive difficulty and support rules

Rule	Condition	Action
Level up	Seconds < 25 and attempts ≤ 1 and hints = 0 and level < HARD	Level + 1; log “Fast solve ... difficulty up”
Level down	Attempts ≥ 4 or hints ≥ 2 or seconds > 90, and level > EASY	Level − 1; log “Player struggled ... difficulty down”
Stay	None of the above	Level unchanged; log “Difficulty stays ...”
Struggle bonus	Attempts ≥ 3 or more than 60 s in the puzzle (checked every second and after each wrong answer); once per puzzle, stages 1 to 4	Allowed hints + 1; message “The AI noticed you're stuck”

4.4 Module 4: Hint and Struggle Support

The Hint button gives the next hint of the current puzzle and shows how many are left, for example “Hint (2 left)”. Each puzzle allows two hints, and the struggle bonus of Table 4.2 allows a third. The generators prepare their hints from the secret: the Number Lock reveals one of the first three digits, the Object Arrangement reveals the position of one object, and the Logic Doors puzzle explains the method and names one door that is not the exit. In the final stage the hint reminds the player how the answer is formed. Using hints counts against the player in the adaptive rule.

4.5 Module 5: Game UI and Inventory

- **Start overlay** with the title, a short story and the buttons “Enter the room” and “How does the AI work?”.
- **Heads-up display** with the game timer, the current AI difficulty and the algorithm name, and a four-step progress bar that marks stages as done, current or waiting.
- **Puzzle area** with the objective, the clue list and the controls of the stage: a keypad with backspace and OK for the Number Lock, click-to-select and click-to-place objects for the arrangement, and three door buttons for the Logic Doors.
- **Inventory** that lists the lock code, the key position and the exit door as they are found.
- **Feedback** through a green pulse for success and a shake for a wrong answer, a message line, and a layout that becomes one column below 800 px width. Animations are switched off for users who prefer reduced motion.

4.6 Module 6: Final Escape and Victory

The last stage asks the player to type the lock code, the exit door letter and the key position in one string, for example 9203A2. The game compares it with the items collected, as in Figure 4.3. A correct answer stops the timer and opens the victory screen, which shows the total time, the number of hints used and the final AI difficulty, and starts a confetti animation. “Play again” reloads the page, which creates a completely new game.

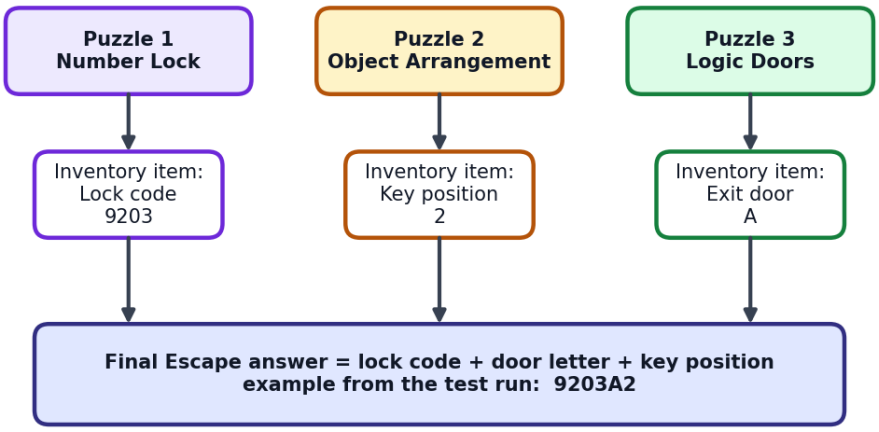


Fig. 4.3 How the three puzzle items form the final answer

4.7 Module 7: AI Explanation and Status

The AI Explanation overlay can be opened from the start screen and from the heads-up display. It has six short sections: what a CSP is, what backtracking is, how the game uses

CSPs, how the AI generates and solves puzzles, how constraints are checked, and how adaptive difficulty works. It ends with a small flow diagram: variables, possible values, constraint check, and then “continue” if the values are valid or “backtrack” if they are not. The AI Status panel on the game screen shows the counters `stats.backtracks` and `stats.checks`, which are reset at the start of each puzzle and refreshed every half second, together with the five latest messages of the AI log.

CHAPTER 5
IMPLEMENTATION AND RESULTS

5.1 Experimental Setup

The game was implemented as three files (`index.html`, `style.css` and `script.js`) and is started by opening `index.html` in a web browser. To measure the AI engine, the unchanged `script.js` was loaded in a headless Chromium browser that was driven by a Playwright script, and the browser's `performance.now()` clock was used to time puzzle generation. The same script played the game from start to finish to capture the output screens of Section 5.2. Table 5.1 lists the environment.

Table 5.1 Software environment

Component	Version / tool
Languages	HTML5, CSS3, JavaScript (ES6)
Game files	index.html, style.css, script.js
Browser used for the experiments	Chromium 141.0.7390.37 (headless)
Automation	Playwright for Python (timing, answer checking and screenshots)
Server, database, external libraries	None

Table 5.2 describes the experiment. For every puzzle type and level, 200 puzzles were generated. For each puzzle the number of clues, the backtracks, the constraint checks and the generation time were recorded. Afterwards the final constraint set of each puzzle was solved again by a separate call with a limit of five solutions, to confirm that exactly one solution exists, and the stored answer was tested against every constraint. The backtrack and check counts include all the work done while one puzzle is generated: choosing the secret, every uniqueness test and the final solve.

Table 5.2 Experiment configuration

Parameter	Value
Puzzle types	Number Lock, Object Arrangement, Logic Doors
Difficulty levels	EASY, MEDIUM, HARD (Number Lock and Object Arrangement)
Puzzles generated	200 per type and level: 1,400 in total
Measured values	Clues, backtracks, constraint checks, generation time
Verification	Independent solution count (limit 5) and test of the stored answer

5.2 Results

5.2.1 Start Screen

Figure 5.1 shows the start screen. It names the course topic and the algorithm, tells the short story and offers two buttons: one enters the room and one opens the AI Explanation.

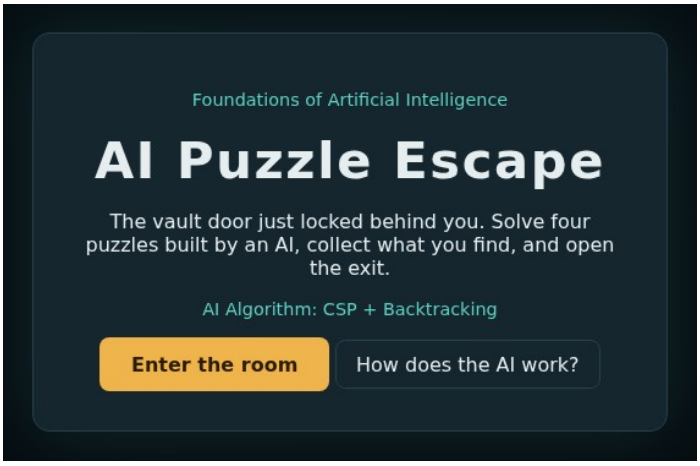


Fig. 5.1 Start screen of AI Puzzle Escape

5.2.2 AI Explanation

Figure 5.2 shows the AI Explanation overlay with the six sections and the flow diagram, which the player can read before or during the game.

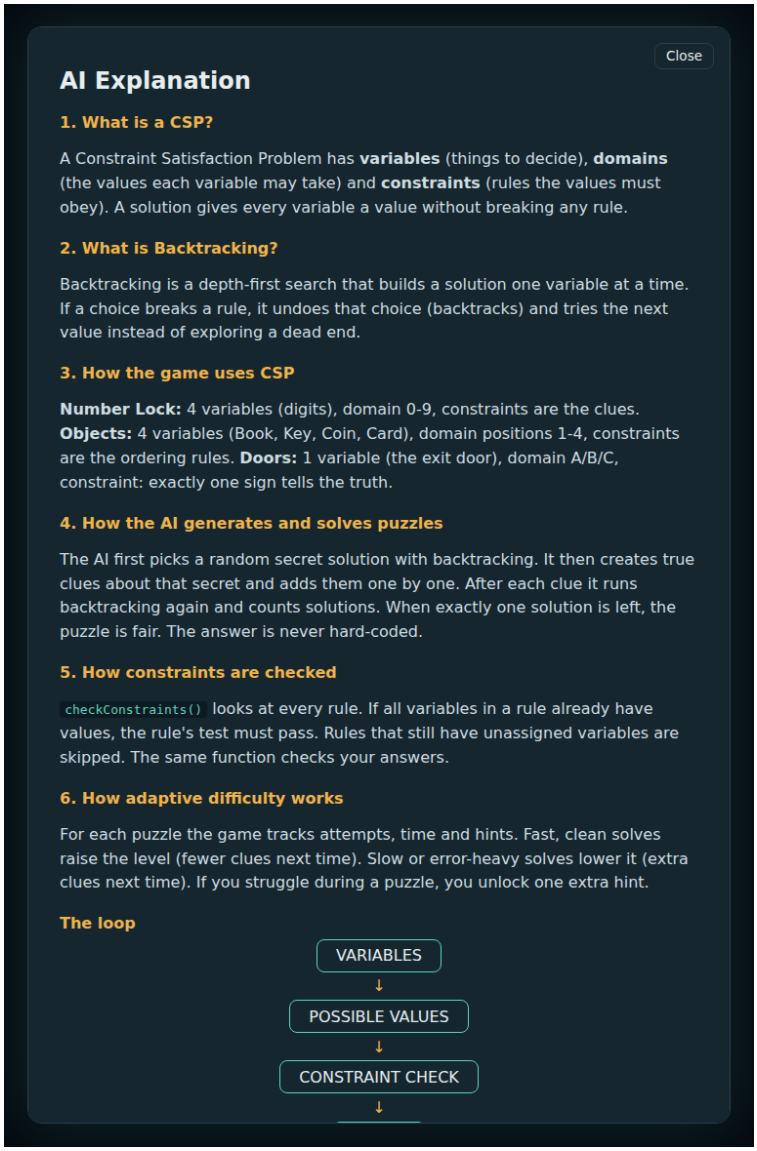


Fig. 5.2 AI Explanation overlay

5.2.3 Puzzle 1: Number Lock

Figure 5.3 shows the first puzzle at MEDIUM difficulty. The clue list was written by the AI for a random secret code, and the first line states that no digit is repeated. The AI Status panel on the right shows the backtracks and constraint checks used to generate this puzzle and the first entry of the AI log. A hint has been taken and is shown below the keypad.

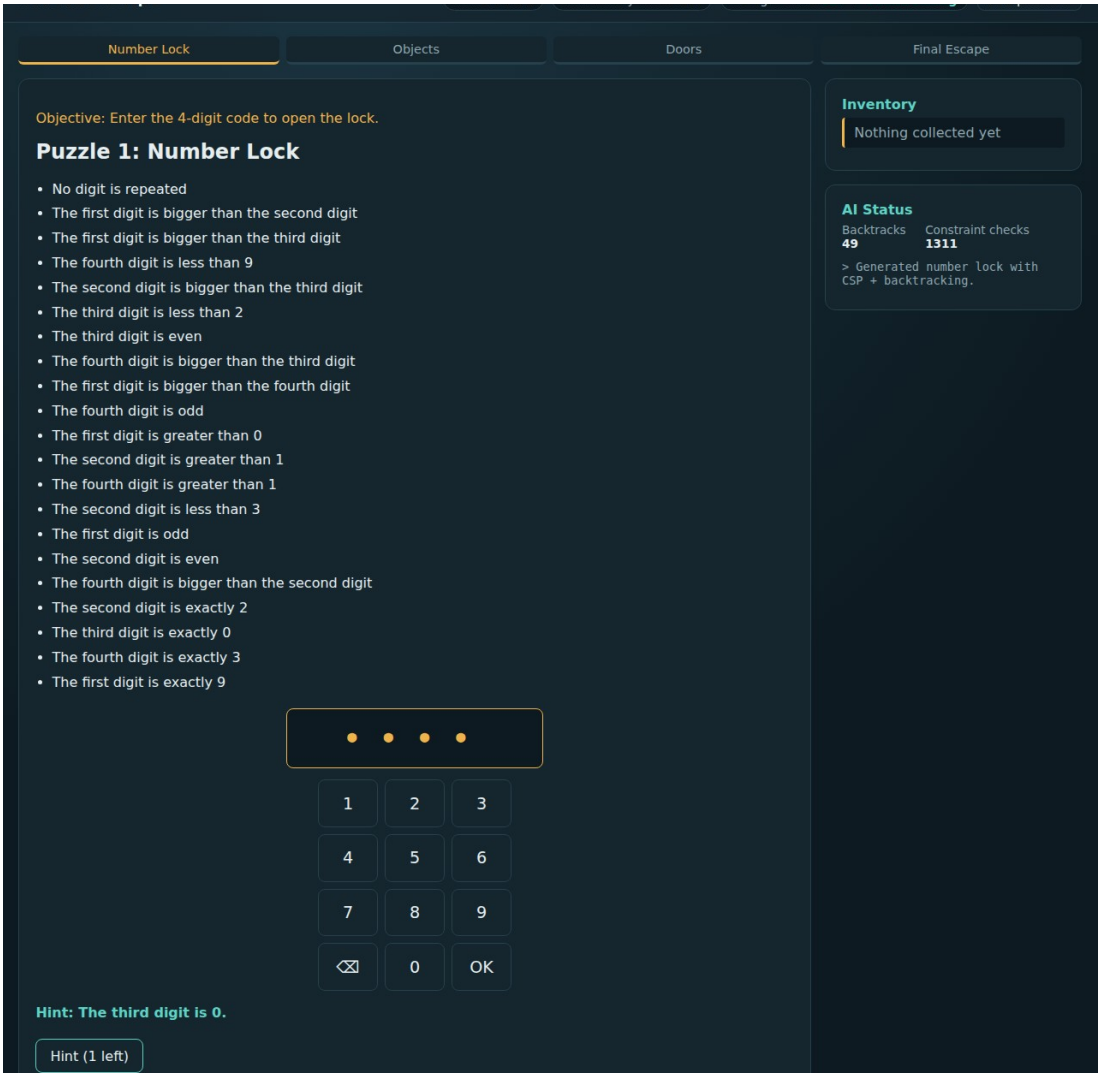


Fig. 5.3 Puzzle 1: Number Lock with AI Status panel

5.2.4 Puzzle 2: Object Arrangement

Figure 5.4 shows the arrangement puzzle after the Number Lock was solved: the inventory already holds the lock code, and the AI log records that the difficulty stays MEDIUM. The Key and the Book have been placed in positions 2 and 4, and the Coin and the Card are still in the tray. The seven clues are all satisfied by exactly one arrangement.

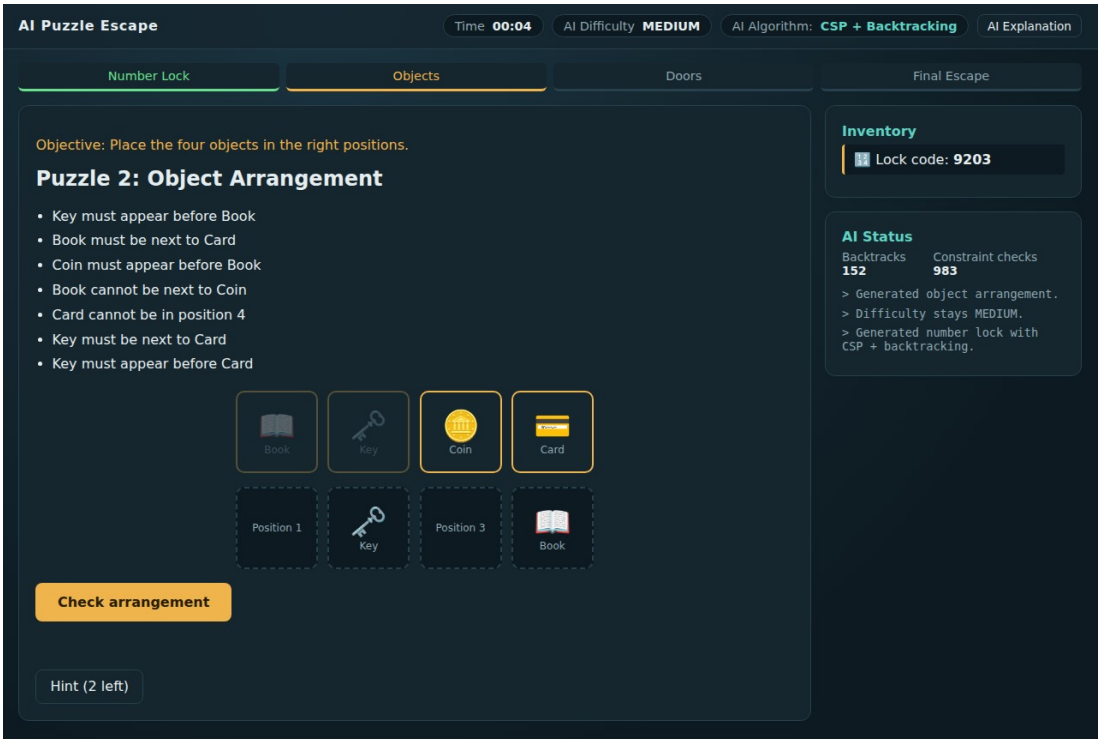


Fig. 5.4 Puzzle 2: Object Arrangement

5.2.5 Puzzle 3: Logic Doors

Figure 5.5 shows the Logic Doors puzzle after the player clicked a wrong door. In this puzzle the exit is Door A: if Door A is the exit, only the sign on Door A is true; if Door B were the exit, the signs on Doors B and C would both be true, and if Door C were the exit, no sign would be true. Only Door A therefore gives exactly one true sign. The game explains the mistake and tells the player how to reason. The log also shows that a fast solve in the previous stage raised the difficulty to HARD.

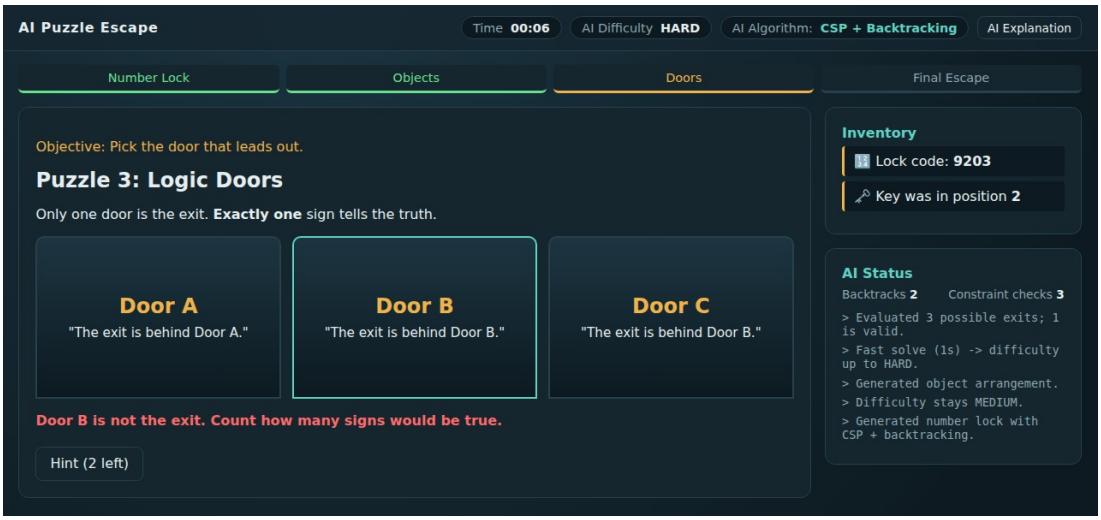


Fig. 5.5 Puzzle 3: Logic Doors after a wrong choice

5.2.6 Final Escape and Victory

Figure 5.6 shows the final stage with the three items in the inventory, and Figure 5.7 shows the victory screen with the total time, the hints used and the final difficulty.

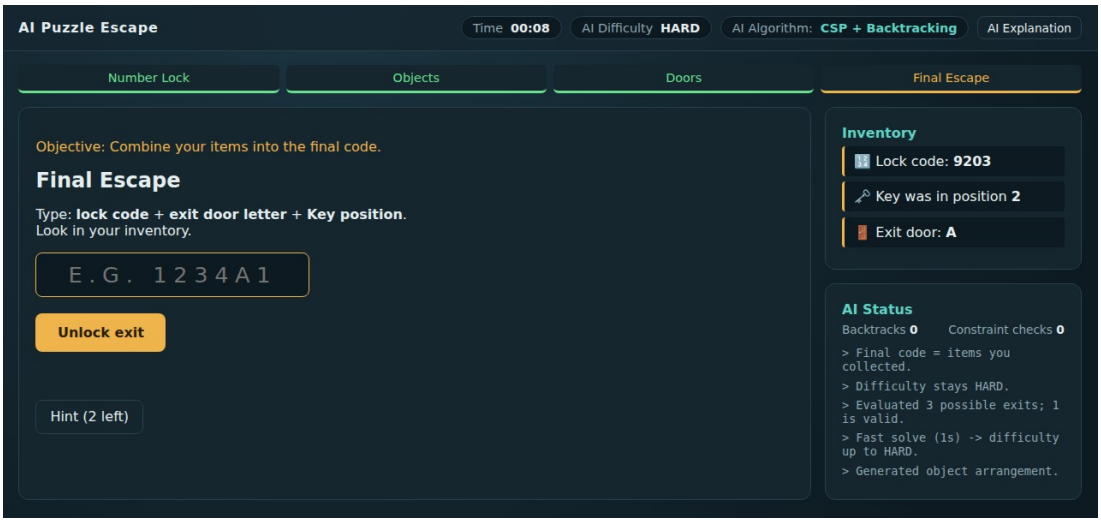


Fig. 5.6 Final Escape stage with collected inventory

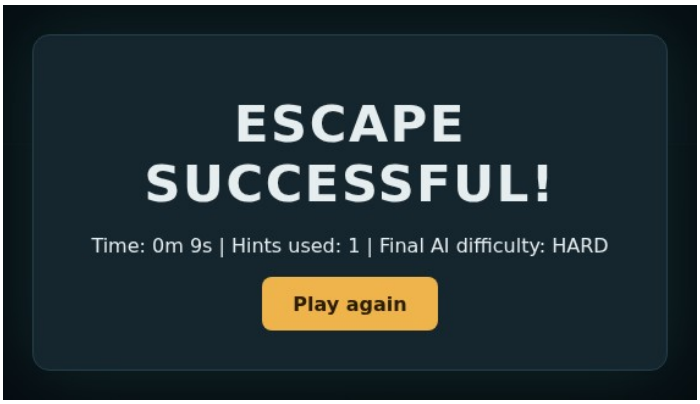


Fig. 5.7 Victory screen

5.2.7 Measurements of the AI Engine

Tables 5.3 to 5.5 give the measured results. In all 1,400 generated puzzles exactly one solution existed and the stored answer satisfied all constraints.

Table 5.3 Number Lock results (200 puzzles per level)

Level	Clues (mean)	Clues (min – max)	Backtracks (mean)	Checks (mean)	Time ms (mean / max)	Unique / valid
EASY	21.4	16 – 23	192.4	2422	1.04 / 14.5	200 / 200 of 200
MEDIUM	21.0	13 – 23	219.5	2629	0.27 / 5.1	200 / 200 of 200
HARD	20.3	12 – 23	209.9	2569	0.10 / 0.4	200 / 200 of 200

Table 5.4 Object Arrangement results (200 puzzles per level)

Level	Clues (mean)	Clues (min – max)	Backtracks (mean)	Checks (mean)	Time ms (mean / max)	Unique / valid
EASY	8.6	5 – 14	154.8	946	0.08 / 4.1	200 / 200 of 200
MEDIUM	7.3	4 – 14	142.8	881	0.06 / 1.3	200 / 200 of 200
HARD	6.9	3 – 14	144.7	909	0.07 / 3.1	200 / 200 of 200

Table 5.5 Logic Doors results (200 puzzles)

Measure	Observed value
Raw search space	3 possible exits
Backtracks per generation (mean)	3.7
Constraint checks per generation (mean)	5.4
Generation time (mean / maximum)	0.02 ms / 1.5 ms
Exactly one solution, equal to the stored exit	200 of 200

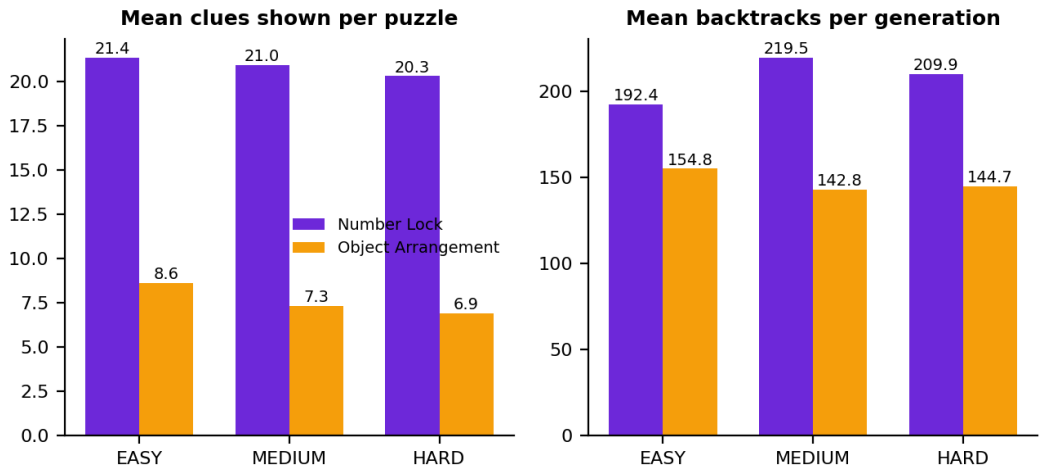


Fig. 5.8 Measured clues and backtracks per difficulty level

5.2.8 Discussion

The results show that the generate-and-verify method works: all 1,400 puzzles had a unique solution, the stored answers satisfied every clue, and generation was fast. The mean generation time was between 0.02 ms and 1.04 ms, and the slowest single generation took

14.5 ms, so the player never waits for a puzzle. The EASY Number Lock mean of 1.04 ms is the first batch measured and probably includes the browser's warm-up of the JavaScript engine, so the times should be read as an order of magnitude, not as exact values.

The measurements also show weaknesses. The Number Lock puzzles show about 20 clues on average (range 12 to 23), which is a long list for a four-digit code. The reason is that the builder adds clues in random order and stops only when the solution is unique, and many clues, such as “the first digit is odd”, remove only a few possibilities. The effect of the difficulty level is also small: between EASY and HARD the mean number of clues fell by about one for the Number Lock (21.4 to 20.3) and by about 1.7 for the Object Arrangement (8.6 to 6.9). This is expected, because the level can change the count only by the extra clues (2, 1 or 0), and the random order of the pool adds more variation than this.

The results have limits. They come from one machine and one browser engine, the play-through was automated, and no study with real players has been done, so the thresholds of the adaptive rules (25 s, 90 s, 3 and 4 attempts, 2 hints) were chosen by the developers and have not been tested on players. The engine uses plain backtracking without the MRV heuristic or forward checking, which was sufficient for these small search spaces but would have to be improved for larger puzzles.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This project built a complete escape-room game around a classical AI method. Each puzzle was modelled as a Constraint Satisfaction Problem and solved with backtracking. The engine chooses a random secret, writes true clues about it, and adds the clues until exactly one solution remains, so no answer is hard-coded and every puzzle is fair. The same constraint checker validates the player's answers. In an experiment with 1,400 generated puzzles all of them had exactly one solution, and the mean generation time was about one millisecond or less.

Around the engine, the game provides a four-stage escape with an inventory and a final combined code, an adaptive difficulty module based on time, attempts and hints, a struggle detector that unlocks an extra hint, and an AI Status panel and AI Explanation page that make the algorithm visible. The work shows that a small classical algorithm can both create and judge puzzles in a playable game and still be explained to the player.

6.2 Future Work

- Reduce the number of clues in the Number Lock by choosing the most informative clues first or by removing clues that are not needed after the puzzle is unique.
- Make the difficulty level change the puzzles more strongly, for example by using weaker clue types or larger domains at higher levels.
- Add the Minimum Remaining Values heuristic and forward checking to the engine and compare the backtrack counts with the current version.
- Add more puzzle types, such as sequences, grid puzzles and word locks, and more rooms with a story.
- Test the adaptive rules with real players and tune the thresholds, or replace them with a learned player model.
- Save progress and best times in the browser, and add sound and accessibility features such as keyboard play and screen-reader labels.

CHAPTER 7

REFERENCES

1. Brailsford, S.C., Potts, C.N. and Smith, B.M. (1999) ‘Constraint satisfaction problems: Algorithms and applications’, European Journal of Operational Research, Vol.119, No.3, pp.557-581.
2. Csikszentmihalyi, M. (1990) Flow: The Psychology of Optimal Experience, Harper & Row, New York.
3. Dechter, R. (2003) Constraint Processing, Morgan Kaufmann, San Francisco.
4. Golomb, S.W. and Baumert, L.D. (1965) ‘Backtrack Programming’, Journal of the ACM, Vol.12, No.4, pp.516-524.
5. Haralick, R.M. and Elliott, G.L. (1980) ‘Increasing Tree Search Efficiency for Constraint Satisfaction Problems’, Artificial Intelligence, Vol.14, No.3, pp.263-313.
6. Hunicke, R. (2005) ‘The Case for Dynamic Difficulty Adjustment in Games’, Proceedings of the ACM SIGCHI International Conference on Advances in Computer Entertainment Technology, pp.429-433.
7. Mackworth, A.K. (1977) ‘Consistency in Networks of Relations’, Artificial Intelligence, Vol.8, No.1, pp.99-118.
8. Nicholson, S. (2015) ‘Peeking Behind the Locked Door: A Survey of Escape Room Facilities’, white paper, Meaningful Play.
9. Russell, S. and Norvig, P. (2020) Artificial Intelligence: A Modern Approach, 4th edition, Pearson, Hoboken.
10. Simonis, H. (2005) ‘Sudoku as a Constraint Problem’, Proceedings of the 4th International Workshop on Modelling and Reformulating Constraint Satisfaction Problems, pp.13-27.
11. Smullyan, R. (1978) What Is the Name of This Book? The Riddle of Dracula and Other Logical Puzzles, Prentice-Hall, Englewood Cliffs.
12. Tsang, E. (1993) Foundations of Constraint Satisfaction, Academic Press, London.